

Chapitre 6 – Android - Synthèse

Sommaire

I. Les tâches à accomplir	1
II. Réalisation.....	1

En guise de synthèse, on va développer une application capable de fournir sur un mobile Android la liste des dix communes les plus proches de la position actuelle.

I. Les tâches à accomplir

Une liste exhaustive des communes françaises (environ 35000) est disponible au format XLS. Une base sera créée, et l'application Android devra proposer la liste des villes les plus proches géographiquement, grâce aux coordonnées GPS de chaque ville et celles du mobile. Cette liste (triée) sera limitée aux 10 ou 20 premières villes (à voir).

- Base interne ou externe ?
- Maquetter l'interface Android ;
- Créer la base MySQL ;
- Importer dans la table les données ;
- Ecrire le script PHP, et le tester ;
- Ecrire l'application Android ;
- Pour les premiers à finir, petit plus :
 - Indiquer dans la liste la distance approximative (pour info : 1°=111 Km à l'équateur) ;
 - Faire passer la liste à 30 villes au lieu de 20 ;
 - Ecrire en plus petit sur le mobile.

II. Réalisation

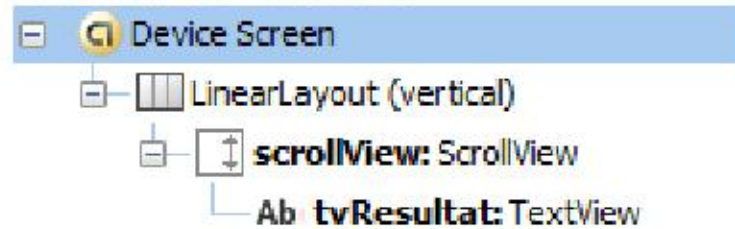
1. Base interne ou externe ?

Intérêt d'une base interne : Même sans accès internet, cela fonctionne. Pour une utilisation réelle, il faut si la base est externe que le serveur web ait un nom de domaine public...

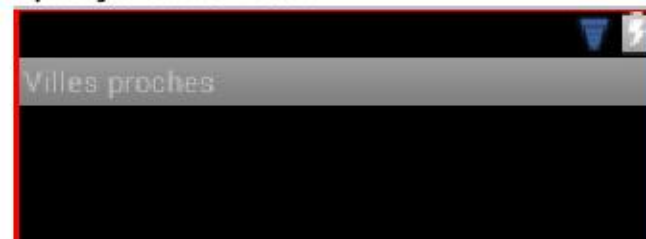
Intérêt de la base externe : La performance... 35000 villes dans un Smartphone ... Il n'est pas fait pour cela !

2. L'application – L'interface

Eléments de l'interface



Aperçu dans *Idea*



Aperçu de l'exécution



3. BaseMySQL

Table tblVilleGPS :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	<u>id</u>	int(11)			Non	Aucune	AUTO_INCREMENT
2	nomVille	varchar(30) latin1_swedish_ci			Non	Aucune	
3	nomVilleMAJ	varchar(30) latin1_swedish_ci			Non	Aucune	
4	codePostal	varchar(5) latin1_swedish_ci			Non	Aucune	
5	latitude	float			Non	Aucune	
6	longitude	float			Non	Aucune	

```

CREATE TABLE IF NOT EXISTS `tblvillegps` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nomVille` varchar(30) NOT NULL,
  `nomVilleMAJ` varchar(30) NOT NULL,
  `codePostal` varchar(5) NOT NULL,
  `latitude` float NOT NULL,
  `longitude` float NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
  
```

4. Importation des données XLS

A partir du fichier des villes françaises au format *xlsx*,

- Créer un fichier via *Notepad++* nommé *Villes de France et GPS.txt* ;
- Pour gérer correctement les caractères, **encoder ce fichier en UTF-8** ;
- Prendre toutes les données d'*Excel* (6 colonnes X 35249 lignes), **sans les entêtes** ;
- Les copier/coller dans *NotePad++* ;

- Enregistrer et quitter *NotePad++* ;
- Avec *PHPMyAdmin*, créer dans la base *bdVille* de MySQL la table *tblVilleGPS* (*id*, *nomVille*, *nomVilleMAJ*, *codePostal*, *latitude*, *longitude*).
- Importer dans cette table les données du fichier .txt :
 - Pointer le fichier *Villes de France et GPS.txt* ;
 - Définir l'encodage à *UTF-8* ;
 - Choisir le format *CSV* ;
 - Modifier le séparateur de colonne (tabulation) avec *\t* ;
 - Supprimer le guillemet entourant les colonnes ;
 - Valider.

Patienter quelques dizaines de secondes, car le fichier de 35000 villes est envoyé au serveur web (1,7 Mo), puis traité.

Vérification des données dans PhpMySQL :

id	nomVille	nomVilleMAJ	codePostal	latitude	longitude
1	Bourg-en-Bresse	BOURG EN BRESSE	01000	46.2	5.21667
2	Saint-Denis-lès-Bourg	SAINT DENIS LES BOURG	01000	46.2	5.2
3	Amareins	AMAREINS	01090	46.0833	4.8
4	Cesseins	CESSEINS	01090	46.0667	4.83333
5	Amareins Francheleins Ces	AMAREINS FRANCHELEINS CES	01090	46.0667	4.8

5. Service web en php

Comme vu précédemment, le format d'échange utilisé peut être le texte brut ou le format JSON. La solution la plus simple (texte brut) est présentée ici.

Pour trouver les villes les plus proches, la latitude et la longitude sont passés en paramètre, et un tri est réalisé sur l'expression $(latitudeVille - latitudeGPS)^2 + (longitudeVille - longitudeGPS)^2$. Les 20 premières villes sont affichées :

```
<?php
try {
    $pdo=new PDO("mysql:host=127.0.0.1;dbname=villes;charset=utf8","root","root");
} catch (PDOException $e) {
    die ("Echec lors de la connexion à MySQL : ".$e->getMessage());
};

$lat=$_REQUEST['lat'];
$long=$_REQUEST['long'];

$req="SELECT nomVille, SQRT (POWER(latitude-:lat,2)+POWER(longitude-:long,2))
as ecart FROM ville_gps ORDER BY ecart LIMIT 0,30";
$stmt=$pdo->prepare ($req);
$stmt->bindParam (':lat',$lat);
$stmt->bindParam (':long',$long);
$stmt->execute () or showErrorsLearning ($pdo);
$lesCommunes=$stmt->fetchAll ();

foreach ($lesCommunes as $uneCommune) {
    $distance=round (111*$uneCommune ["ecart"]); // 1° = 111 Kms (à l'équateur)
    $ville=$uneCommune ["nomVille"];
    echo (" $ville - $distance km(s)\n");
};
?>
```


Le problème des accents trouve sa solution dans le codage des données en UTF-8 pour le transfert entre la base et le serveur web.

Test du script PHP :



Alixan Fauconnières Bourg-de-Péage Saint-Marcel-lès-Valence Parlanges Romans-sur-Isère Montélier Saint-Bardoux
Mours-Saint-Eusèbe Chatuzange-le-Goubet Beaumont-Monteux Clérieux Bésayes Peyrins Chabeuil Marches Malissard
Chanos-Curson Pont-de-l'Isère Saint-Donat-sur-l'Herbasse

Données sources reçues :

```
1 Alixan
2 Fauconnières
3 Bourg-de-Péage
4 Saint-Marcel-lès-Valence
5 Parlanges
6 Romans-sur-Isère
7 Montélier
8 Saint-Bardoux
9 Mours-Saint-Eusèbe
10 Chatuzange-le-Goubet
11 Beaumont-Monteux
12 Clérieux
13 Bésayes
14 Peyrins
15 Chabeuil
16 Marches
17 Malissard
18 Chanos-Curson
19 Pont-de-l'Isère
20 Saint-Donat-sur-l'Herbasse
```

6. Application Android : Le code

Comme pour l'application de géo-localisation, cette application utilise le GPS et demande la même initialisation :

```
public class MyActivity extends Activity {
    /**
     * Called when the activity is first created.
     */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        LocationManager locationManager = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
        LocationListener locationListener = new MyLocationListener();
        locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 0, 0, locationListener);
    }
}
```

Quand le GPS détecte un changement de position, il faut lancer le service web *villesproches.php* avec comme paramètres la latitude et la longitude :

```

class MyLocationLister implements LocationListener {

    @Override
    public void onLocationChanged(Location location) {
        TextView resultat;
        if (location != null) {
            String urlServiceWeb;

            double dLat = location.getLatitude();
            double dLong = location.getLongitude();

            resultat = (TextView) findViewById(R.id.tvResultat);
            urlServiceWeb = "http://192.168.0.14/villesproches.php?lat=" + dLat + "&long=" + dLong;
            resultat.setText("Villes proches :\n");
            resultat.append(getServerData(urlServiceWeb));
        }
    }
}

```

Les autres méthodes du GPS restent surchargées sans rien faire, comme dans le cas *Géo-Localisation* :

```

@Override
public void onStatusChanged(String s, int i, Bundle bundle) {
}

@Override
public void onProviderEnabled(String s) {
}

@Override
public void onProviderDisabled(String s) {
}

```

Enfin, l'accès au service web est réalisé de la même manière que dans le chapitre sur les bases de données, avec l'option base externe *MySQL* :


```

private String getServerData(String url) {
    InputStream is = null;
    String ch = "";

// Autoriser les opérations réseau sur le thread principal
StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
StrictMode.setThreadPolicy(policy);

    try {
        HttpClient httpClient = new DefaultHttpClient();
        HttpPost httpPost = new HttpPost(url);
        HttpResponse httpResponse = httpClient.execute(httpPost);
        HttpEntity httpEntity = httpResponse.getEntity();
        is = httpEntity.getContent();

        BufferedReader br = new BufferedReader(new InputStreamReader(is));
        String ligne;
        while ((ligne = br.readLine()) != null) {
            ch = ch + "- " + ligne + "\n";
        }
    } catch (Exception e) {
        Log.e("log_tag", "Erreur pendant la récupération des données : " + e.toString());
    }
    return (ch);
}
}

```

7. Droit spécifique nécessaire

Cette application utilisant le GPS, ne pas oublier de réclamer ce droit dans le fichier *AndroidManifest.xml* :

```

-----
    android:versionName="1.0">
<uses-sdk android:minSdkVersion="16"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<application android:label="@string/app_name" android:icon="@drawable/ic_launcher">
    <activity android:name="MyActivity"

```

Pour ceux qui ont terminé, faire une version avec thread spécifique, et éventuellement avec codage JSON ...